二次规划问题中的 Solver - HQS，ISTA, FISTA, TwIST, ADMM，GAP，OSQP

二次规划问题中的 Solver - HQS，ISTA, FISTA, TwIST, ADMM，GAP，OSQP
- 二次规划 (Quadratic Program，QP) 问题
  - 相关术语
- LASSO问题
- 半二次分裂算法 (Half quadratic splitting，HQS)
  - 变量分裂法（Variable Spliting）
  - 半二次分裂
- （加速）迭代收缩阈值算法 ((Fast) iterative shrinkage-thresholding algorithm (F)ISTA)
  - 近端梯度下降 (Proximal Gradient Algorithm, PGD)
  - 迭代收缩阈值算法（ iterative shrinkage-thresholding algorithm ISTA)
  - 加速迭代收缩阈值算法（fast iterative shrinkage-thresholding algorithm FISTA)
- 两步迭代收缩阈值算法 (Two-step iterative shrinkage/thresholding , TwIST)
- 广义交替投影算法 (Generalized alternating projection, GAP)
- 交替方向乘子法 (Alternating direction method of multipilers, ADMM)
- 算子分裂二次规划求解算法 (Operator splitting solver for quadratic programs, OSQP)
- 附录 Appendix
  - 软阈值算子（soft-thresholding operator）
  - Lipschitz常数和Lipschitz连续
  - Nesterov加速技术
- Reference

二次规划 (Quadratic Program，QP) 问题

二次规划问题是非线性规划（Non-linear program，NLP）问题的特例，即当目标函数 f 为二次型且约束 h，g 在 $x \in \mathbf{R}^n$ 为线性约束时的 NLP 问题即为 QP 问题，其一般形式如下：

$$\begin{aligned} \min \quad & f(x) = \frac{1}{2}\mathbf{x}^\intercal \mathbf{B}\mathbf{x} - \mathbf{x}^\intercal \mathbf{b}, \mathbf{x} \in \mathbb{R}^n \\ \text{s.t.} \quad & \mathbf{A}_1 \mathbf{x} = \mathbf{c} \\ & \mathbf{A}_2 \mathbf{x} \leq \mathbf{d} \end{aligned}$$

上式中，$\mathbf{B} \in \mathbb{R}^{n\times n}$是对称矩阵, $\mathbf{b} \in \mathbb{R}^n$, $\mathbf{A}_1 \in \mathbb{R}^{m\times n}$, $c \in \mathbb{R}^n$, $\mathbf{A}_2 \in \mathbb{R}^{p\times n}$, $\mathbf{d} \in \mathbb{R}^p$

相关术语

- 二次型，正定矩阵
  - 二次型$f(x) = \mathbf{x}^\intercal \mathbf{A}\mathbf{x}$ ，其中对称矩阵 $\mathbb{A} \in \mathbb{R}^{n\times n}, \mathbf{x} \in \mathbb{R}^n$； 二次型 $f(x)$ 和对称矩阵 $\mathbf{A}$ 具有一一对应关系
  - 若$\forall \mathbf{x} \neq 0, f(x) \geq 0$, 则二次型 $f(x)$ 为正定二次型； 正定二次型的充要条件为 其对应的对称矩阵的所有特征值大于0； 或者其对应的对称矩阵的各阶主子式均为正
- 海森（Hessian）矩阵
  - 假设有函数 $f: \mathbb{R}^n \to \mathbb{R}$，如果在定义域中其二阶偏导数存在且连续，那么 f 的 Hessian 矩阵为一个 $n \times n$ 的方阵，定义为 $\mathbf{H}_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$， 根据二阶连续偏导和求导次序无关可知，Hessian 矩阵为对称矩阵。

LASSO问题

LASSO回归问题是在线性回归问题的基础上，加上 l_1 范数约束，来限制线性拟合的系数，保持一定的稀疏性。相比于岭回归 ($l_2 - norm$ 范数约束)，LASSO回归对离群极端值具有较好的鲁棒性。LASSO问题的形式如下：

$$\min_x \quad \frac{1}{2}\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 + \gamma\|\mathbf{x}\|_1$$

其中 $\mathrm{x} \in \mathbb{R}^n$ 为系数向量 , $\mathbb{A} \in \mathbb{R}^{m\times n}$ 是数据矩阵，$\lambda > 0$ 是惩罚项权重。

LASSO问题的等价二次规划问题为

$$\begin{array}{ll} \text{minimize} & \frac{1}{2}\mathbf{y}^T\mathbf{y}+\gamma\mathbf{1}^T t \\ \text{subject to} & \mathbf{y}=\mathbf{A}\mathbf{x}-\mathbf{b} \\ & -\mathbf{t}\leq\mathbf{x}\leq\mathbf{t} \end{array}$$

半二次分裂算法 (Half quadratic splitting，HQS)

变量分裂法（Variable Spliting)

用于解决目标函数是两个函数之和的优化问题，如假设 g 是 n 维向量到 d 维向量的一个映射，优化目标函数为：

$$\min_{u\in\mathbb{K}^n} \quad f_1(\mathbf{u})+f_2(g(\mathbf{u}))$$

可以通过变量分裂将上式变为：

$$\begin{array}{ll} \min\limits_{u\in\mathbb{K}^n} & f_1(\mathbf{u})+f_2(\mathbf{v}) \\ \text{s.t.} & g(\mathbf{u})=\mathbf{v} \end{array}$$

变量分裂之后可能比原问题更好解决。

半二次分裂

通常是将正则项中的原始变量进行变量替换，而后通过增广拉格朗日法（Augmented Lagrange Method，ALM）重写目标函数（包含原数据保真项，拉格朗日乘子项和二次惩罚项，这么做的目的是**解耦的同时，简化计算。** 图像复原中，目标函数为：

$$\hat{\mathbf{x}}=\arg\min_{\mathbf{x}}\frac{1}{2}\|\mathbf{y}-\mathbf{H}\mathbf{x}\|^2+\lambda\Phi(\mathbf{x})$$

前一项为数据保真项（fidelity），后一项为惩罚项，通常只与去噪有关。通过HQS算法，引入辅助变量 z，把惩罚项的 x 替换为 z，并添加相应的等式约束条件：

$$\hat{\mathbf{x}}=\arg\min_{\mathbf{x}}\frac{1}{2}\|\mathbf{y}-\mathbf{H}\mathbf{x}\|^2+\lambda\Phi(\mathbf{z}) \quad \text{s.t.} \quad \mathbf{z}=\mathbf{x}$$

由增广拉格朗日法（Augmented Lagrange Method，ALM）得到增广拉格朗日函数：

$$\mathcal{L}_\mu(\mathbf{x},\mathbf{z})=\frac{1}{2}\|\mathbf{y}-\mathbf{H}\mathbf{x}\|^2+\lambda\Phi(\mathbf{z})+\frac{\mu}{2}\|\mathbf{z}-\mathbf{x}\|^2$$

迭代求解：

$$\begin{array}{l} \mathbf{x}_{k+1}=\arg\min_{\mathrm{x}}\|\mathbf{y}-\mathbf{H}\mathbf{x}\|^2+\mu\|\mathbf{x}-\mathbf{z}_k\|^2 \\ \mathbf{z}_{k+1}=\arg\min_{\mathrm{z}}\frac{\mu}{2}\|\mathbf{z}-\mathbf{x}_{k+1}\|^2+\lambda\Phi(\mathbf{z}) \end{array}$$

两个子问题将原问题的保真项和先验项解耦分别求解。

（加速）迭代收缩阈值算法 ((Fast) iterative shrinkage-thresholding algorithm (F)ISTA)

近端梯度下降 (Proximal Gradient Algorithm, PGD)

对于包含不可微部分的优化目标如lasso回归的求解，梯度下降等算法已不再适用。近端梯度下降即被用来解决包含不可导项的优化问题。研究如下问题：

$$\min_x \quad g(\mathbf{x})+h(\mathbf{x})$$

其中，$g(x)$是连续可导凸函数；$h(x)$是连续的凸函数，可以是不光滑的（不可导，not necessarily differentiable）。对于函数 $f(x)$，将其在x_0 处做二阶泰勒展开，将其用来近似原函数得：

$$f(\mathbf{x})\approx f(\mathbf{x}_0)+\nabla f(\mathbf{x}_0)^T(\mathbf{x}-\mathbf{x}_0)+\frac{1}{2}(\mathbf{x}-\mathbf{x}_0)^T\nabla^2 f(\mathbf{x})(\mathbf{x}-\mathbf{x}_0)$$

二阶导相对一阶导要难求很多，并且从时间复杂度的角度上来讲，求二阶导信息即Hessian矩阵要花去许多时间。根据Lipschitz continuous gradient，我们将二阶导用 $\frac{1}{t}I_n$代替，其中 $t>0$，I_n为n阶单位矩阵，则

$$f(\mathbf{x}) \approx f\left(\mathbf{x}_0\right)+\nabla f(\mathbf{x}_0)^T\left(\mathbf{x}-\mathbf{x}_0\right)+\frac{1}{2t}\|(\mathbf{x}-\mathbf{x}_0\|_2^2$$

设原问题最优解为x^+，则带入上式到原问题，并抛掉常数$f(x_0)$，凑平方得

$$\begin{aligned}\mathbf{x}^+ &= \arg\min_{\mathbf{x}}\left\{f\left(\mathbf{x}_0\right)+\nabla f(\mathbf{x}_0)^T\left(\mathbf{x}-\mathbf{x}_0\right)+\frac{1}{2t}\|\mathbf{x}-\mathbf{x}_0\|_2^2+h(\mathbf{x})\right\}\\ &= \arg\min_{\mathbf{x}}\left\{\frac{1}{2t}\|\mathbf{x}-\left(\mathbf{x}_0-t\nabla f\left(\mathbf{x}_0\right)\|_2^2+h(\mathbf{x})\right\}\end{aligned}$$

定义**临近点算子/近端算子(proximal operator)**如下：

$$\operatorname{prox}_{t,h(\cdot)}(\mathbf{z})=\arg\min_{\mathbf{x}}\left\{\frac{1}{2t}\|\mathbf{x}-\mathbf{z}\|_2^2+h(\mathbf{x})\right\}$$

等价定义如下：

$$\operatorname{prox}_{t,h(\cdot)}(\mathbf{z})=\arg\min_{\mathbf{x}}\left\{\frac{1}{2}\|\mathbf{x}-\mathbf{z}\|_2^2+th(\mathbf{x})\right\}$$

利用近端算子可以将原问题的最优解写为:

$$\mathbf{x}^+=\operatorname{prox}_{t,h((\cdot))}(\mathbf{x}_0-t\nabla f(\mathbf{x}_0))$$

写为迭代求解形式（近端梯度下降的核心公式）为：

$$\mathbf{x}^k=\operatorname{prox}_{t,h((\cdot))}(\mathbf{x}^{k-1}-t\nabla f(\mathbf{x}^{k-1}))$$

针对不同的 $h(\cdot)$，如 l_0 范数、l_1 范数、l_2 范数、二次函数等，近端算子都有着对应的闭式解(closed-form)（或者说显式解），下面列出了几个简单的情况:

$h(\cdot)=0$ 时， $\operatorname{prox}_{t,h(\cdot)}(\mathbf{z})=\mathbf{z}$

$h(\cdot)=\|\cdot\|_0$ 时，$\operatorname{prox}_{t,h(\cdot)}(z_i)=\begin{cases} z_i & |z_i|\geq t \\ 0 & \text{otherwise}\end{cases}$，Elementwise hard-thresholding (硬阈值函数) $h(\cdot)=\|\cdot\|_1$ 时，$\operatorname{prox}_{t,h(\cdot)}(z_i)=sign(z_i)\max(|z_i|-t,0)$，Elementwise soft-thresholding (软阈值函数)

$h(\cdot)=\|\cdot\|_2$ 时，$\operatorname{prox}t,h(\cdot)(\mathbf{z})=\mathbf{z}\odot\left[1-\frac{t}{\|\mathbf{z}\|_2}\right]_+$，Elementwise soft-thresholding (软阈值函数)

迭代收缩阈值算法（iterative shrinkage-thresholding algorithm ISTA)

一般的，我们将解决上述含不可导项的优化问题的近端梯度下降算法成为 ISTA算法。具体应用中，ISTA算法狭义上一般指不可导项为l_1 范数的LASSO问题。

LASSO问题中使用了 l_1 范数，故对应 $h(\cdot)=\lambda\|\cdot\|_1$ 的情况，此时的近端梯度下降迭代表达式可以写成：

$$\mathbf{x}^k=\operatorname{prox}_{t,\lambda\|\cdot\|_1}(\mathbf{x}^{k-1}-t\nabla f(\mathbf{x}^{k-1}))=\mathrm{S}_{\lambda t}(\mathbf{x}^{k-1}-t\nabla f(\mathbf{x}^{k-1}))$$

其中 $\mathrm{S}_{\lambda t}(\cdot)$ 称为软阈值算子（soft-thresholding operator），也常写作$\mathrm{soft}_{\lambda t}(\cdot)$，可以看到该算子即为近端算子中 $h(\cdot)=\|\cdot\|_1$ 的情况。

若 z 为标量，则有：

$$\operatorname{prox}_{t,\lambda\|\cdot\|_1}(z)=\mathrm{S}_{\lambda t}(z)=sign(z)\max(|z|-\lambda t,0)=\begin{cases} z-\lambda t & z>\lambda t \\ 0 & -\lambda t\leq z\leq\lambda t \\ z+\lambda t & z<-\lambda t\end{cases}$$

若 $\mathbf{z}=[z_1,\cdots,z_n]^\intercal\in\mathbb{R}^n$ 为向量，则 $\mathrm{S}_{\lambda t}(\mathbf{z})=[\mathrm{S}_{\lambda t}(z_1),\cdots,\mathrm{S}_{\lambda t}(z_n)]^\intercal$，即为每个分量均做软阈值操作。进一步可以推导出如下表达式:

$$
\begin{aligned}
S_{\lambda t}\left(z_i\right) &= \operatorname{sign}(z_i) \max\left(|z_i|-t, 0\right) \\
&= \operatorname{sign}(z_i)\,|z_i| \max\left(\frac{|z_i|-t}{|z_i|}, 0\right) \\
&= z_i \max\left(\frac{|z_i|-t}{|z_i|}, 0\right) \\
&= z_i \max\left(1-\frac{t}{|z_i|}, 0\right) \\
&\therefore S_{\lambda t}(\mathbf{z}) = \mathbf{z} \odot \left[1-\frac{t}{|\mathbf{z}|}\right]_+
\end{aligned}
$$

其中，$|z|$ 为对向量 $\mathbf{z}$ 的每个分量取绝对值后组成的向量；$[\cdot]_+$ 表示取正，即 $\max(\cdot, 0)$; $\mathbf{A} \odot \mathbf{B}$ 表示两个矩阵的Hadamard乘积（element-wise product）。

软阈值迭代算法(ISTA, iterative soft-thresholding algorithm)，即为用软阈值作为包含 l_1 范数的目标函数的“梯度”，使用”梯度下降“来得到最优值的算法。回顾LASSO优化问题，可以得到:

$$
\begin{gathered}
\nabla f(x) = \nabla \frac{1}{2}\|\mathbf{A}\mathbf{x}-\mathbf{b}\|_2 = \mathbf{A}^{\intercal}(\mathbf{A}\mathbf{x}-\mathbf{b}) \\
h(x) = \gamma\|x\|_1
\end{gathered}
$$

所以 ISTA 算法的迭代公式为：

$$
\mathbf{x}^k = \mathrm{S}_{\gamma t}(\mathbf{x}^{k-1} - t\nabla f(\mathbf{x}^{k-1})) = \mathrm{S}_{\gamma t}(\mathbf{x}^{k-1} - t\mathbf{A}^{\intercal}(\mathbf{A}\mathbf{x}^{k-1} - \mathbf{b}))
$$

算法流程如下（该算法流程为一般形式的ISTA算法流程，即近端算子$p_L(\cdot)$为不可导项$h(\cdot)$对应的近端算子，特殊的，当不可导项$h(\cdot)$为l_1范数时，$p_L(\cdot) = S_{rt}(\cdot)$，下同）：

ISTA with constant stepsize
Input: $L := L(f)$ - A Lipschitz constant of ∇f.
Step 0. Take $\mathbf{x}_0 \in \mathbb{R}^n$.
Step k. $(k \geq 1)$ Compute

$$
\mathbf{x}_k = p_L(\mathbf{x}_{k-1}).
$$

ISTA算法还有一种推广形式的迭代公式：

$$
\mathbf{x}^k = (1-\beta)\mathbf{x}^{k-1} + \beta\,\mathrm{S}_{\gamma t}(\mathbf{x}^{k-1} - \mathbf{A}^{\intercal}(\mathbf{A}\mathbf{x}^{k-1} - \mathbf{b}))
$$

其中 $\beta > 0$。原始 ISTA 算法的迭代形式可以看做 $\beta = 1$的特例。当 $\beta > 1$ 或者 $\beta < 1$ 时可以将推广形式看做原始形式的 over/under related 版本。

文献中已经证明，当迭代步长取f的Lipschitz常数的倒数（即 $\frac{1}{L(f)}$ ）时，由ISTA算法生成的序列$\mathbf{x_k}$ 的收敛速度为 $\mathcal{O}(\frac{1}{k})$ 显然为次线性收敛速度。然而固定步长的ISTA的缺点是：Lipschitz常数$L(f)$不一定可知或者可计算。例如，L1范数约束的优化问题，其Lipschitz常数依赖于$\mathbf{A}^{\intercal}\mathbf{A}$的最大特征值。而对于大规模的问题，非常难计算。因此出现了ISTA算法的Backtracking版本，通过不断收缩迭代步长的策略使其收敛，算法流程如下：

ISTA with backtracking

Step 0. Take $L_0 > 0$, some $\eta > 1$, and $\mathbf{x}_0 \in \mathbb{R}^n$.

Step k. $(k \geq 1)$ Find the smallest nonnegative integers i_k such that with $\bar{L} = \eta^{i_k} L_{k-1}$

$$F(p_{\bar{L}}(\mathbf{x}_{k-1})) \leq Q_{\bar{L}}(p_{\bar{L}}(\mathbf{x}_{k-1}), \mathbf{x}_{k-1}). \tag{3.2}$$

Set $L_k = \eta^{i_k} L_{k-1}$ and compute

$$\mathbf{x}_k = p_{L_k}(\mathbf{x}_{k-1}).$$

加速迭代收缩阈值算法（fast iterative shrinkage-thresholding algorithm FISTA）

为了加速ISTA算法的收敛，文献中作者采用了著名的梯度加速策略[Nesterov加速技术]，使得ISTA算法的收敛速度从$\mathcal{O}(\frac{1}{k})$变成$\mathcal{O}(\frac{1}{k^2})$，加速的ISTA算法称为FISTA。FISTA与ISTA算法相比，仅仅多了个Nesterov加速步骤，以极少的额外计算量大幅提高了算法的收敛速度。而且不仅在FISTA算法中，在几乎所有与梯度有关的算法中，Nesterov加速技术都可以使用。FISTA的算法流程如下：

FISTA with constant stepsize

Input: $L = L(f)$ - A Lipschitz constant of ∇f.

Step 0. Take $\mathbf{y}_1 = \mathbf{x}_0 \in \mathbb{R}^n$, $t_1 = 1$.

Step k. $(k \geq 1)$ Compute

$$\mathbf{x}_k = p_L(\mathbf{y}_k), \tag{4.1}$$

$$t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}, \tag{4.2}$$

$$\mathbf{y}_{k+1} = \mathbf{x}_k + \left(\frac{t_k - 1}{t_{k+1}}\right)(\mathbf{x}_k - \mathbf{x}_{k-1}). \tag{4.3}$$

当然，考虑到与ISTA同样的问题：问题规模大的时候，决定步长的 Lipschitz 常数计算复杂。FISTA与ISTA一样，亦有其回溯算法。在这个问题上，FISTA与ISTA并没有区别，上面也说了，FISTA与ISTA的区别仅仅在于每一步迭代时近似函数起始点的选择。更加简明的说：FISTA用一种更为聪明的办法选择序列$\{\mathbf{x}_k\}$，使得其基于梯度下降思想的迭代过程更加快速地趋近问题函数$f(x)$的最小值。

带回溯的FISTA算法基本迭代步骤如下：

FISTA with backtracking

Step 0. Take $L_0 > 0$, some $\eta > 1$, and $\mathbf{x}_0 \in \mathbb{R}^n$. Set $\mathbf{y}_1 = \mathbf{x}_0, t_1 = 1$.

Step k. $(k \geq 1)$ Find the smallest nonnegative integers i_k such that with $\bar{L} = \eta^{i_k} L_{k-1}$

$$F(p_{\bar{L}}(\mathbf{y}_k)) \leq Q_{\bar{L}}(p_{\bar{L}}(\mathbf{y}_k), \mathbf{y}_k).$$

Set $L_k = \eta^{i_k} L_{k-1}$ and compute

$$\begin{aligned} \mathbf{x}_k &= p_{L_k}(\mathbf{y}_k), \\ t_{k+1} &= \frac{1+\sqrt{1+4t_k^2}}{2}, \\ \mathbf{y}_{k+1} &= \mathbf{x}_k + \left(\frac{t_k - 1}{t_{k+1}}\right)(\mathbf{x}_k - \mathbf{x}_{k-1}). \end{aligned}$$

值得注意的是，在每一步迭代中，计算近似函数的起止点时，FISTA使用前两次迭代过程的结果$\mathbf{x}_k, \mathbf{x}_{k-1}$，对其进行简单的线性组合生成下一次迭代的近似函数起始点yk。方法很简单，但效果却非常好。当然，这也是有理论支持的。关于ISTA和FISTA的具体理论推导可以看原论文。

两步迭代收缩阈值算法 (Two-step iterative shrinkage/thresholding , TwIST)

TwIST 算法是 ISTA 算法的改进算法，其two-step是指该算法的每次迭代都依赖于前两次迭代的结果，而不仅是前一次迭代的结果。**(F)ISTA算法的收敛速度对线性观测矩阵具有很强的依赖性，当观测矩阵十分病态（ill-posed）时，该算法的收敛速度会很慢。**TwIST通过引入两步（或者说二阶， second order）非线性迭代策略克服了ISTA收敛速度的问题，在病态问题的求解中具有更快的收敛速度。 定义优化目标函数：

$$\min_x \quad \frac{1}{2}\|\mathbf{y} - \mathbf{K}\mathbf{x}\|^2 + \lambda\Phi(\mathbf{x})$$

该目标函数是线性逆问题求解中的常规形式。其中，y是观察数据，K是一个线性操作符，$\Phi(\mathbf{x})$是一个正则项。该目标函数的第一项为数据保真项，第二项为先验项，二者通过正则化参数 λ 控制权重。

TwIST算法的迭代公式为：

$$\begin{aligned} \mathbf{x}_1 &= \Gamma_\lambda(\mathbf{x}_0) \\ \mathbf{x}_{t+1} &= (1-\alpha)\mathbf{x}_{t-1} + (\alpha - \beta)\mathbf{x}_t + \beta\Gamma_\lambda(\mathbf{x}_t) \end{aligned}$$

对于 $t \geq 1$，$\Gamma_\lambda : \mathbb{R}^m \to \mathbb{R}^m$定义为

$$\Gamma_\lambda(\mathbf{x}) = \Psi_\lambda(\mathbf{x} + \mathbf{K}^\intercal(\mathbf{y} - \mathbf{K}\mathbf{x}))$$

其中 Ψ_λ （去噪函数，Moreau proximal mapping）定义为

$$\Psi_\lambda(y) = \arg\min_x \left\{\frac{1}{2}\|\mathbf{x} - \mathbf{y}\|^2 + \lambda\Phi(\mathbf{x})\right\}$$

当 $\Phi(\mathbf{x}) = \|\mathbf{x}\|_1$ 时，$\Psi_\lambda(y) = \mathrm{S}_\lambda(y) = sign(z)\max(|z| - \lambda, 0)$ 即为软阈值函数。更一般的，当$\Phi(\mathbf{x})$ 时为全变分（total variation, TV）、加权 l_p 范数以及加权 l_p 范数的 $p\mathrm{th}$ 次方时对应的 Ψ_λ的推导见原论文。

广义交替投影算法 (Generalized alternating projection, GAP)

考虑优化问题：

$$\min_{\mathbf{w}, C} \quad C \quad \text{s.t.} \quad \|\mathbf{w}\|_{\ell_{2,1}^{\mathcal{G}\beta}} \leq C \quad \text{and} \quad \mathbf{\Phi}\mathbf{w} = \mathbf{y}$$

该优化问题的本质为寻找一个和给定的线性流形（优化函数第二项）有非空交集的最小加权$\ell_{2,1}$范数球（优化函数第一项）。可以通过一系列交替投影求解该问题，引入辅助变量θ 将该问题先进行转化：

$$\left(\mathbf{w}^{(t)}, \boldsymbol{\theta}^{(t)}\right) = \arg\min_{\mathbf{w},\theta} \frac{1}{2}\|\mathbf{w}-\boldsymbol{\theta}\|_2^2 + \lambda^{(t)}\|\theta\|_{\ell_{2,1}^{\mathcal{G}\beta}} \quad \text{subject to} \quad \mathbf{\Phi}\mathbf{w} = \mathbf{y}$$

其中，$\lambda^{(t)} \geq 0$ 是与 $C^{(t)}$ 有关的Lagrangian乘子。通过对 $\mathbf{w}, \theta$进行交替更新可以求解上述优化问题。具体来说，给定 θ 时，$\mathbf{w}$的更新是 θ 在线性流形上的一个Euclidean projection； 给定$\mathbf{w}$，θ 的更新可以通过对$\mathbf{w}$进行groupwise的shrinkage（软阈值操作）来获得，两种情况下的更新都有解析（closed-form）解。具体迭代公式如下（假设 $\mathbf{\Phi}\mathbf{\Phi}^\top$ 可逆）:

$$\begin{aligned}
&(\text{update of } \lambda) && \lambda^{(t)} = \left\|\mathbf{w}_{\mathcal{G}_{j_{m^\star+1}^{(t)}}}\right\|_2 \beta_{j_{m^\star+1}^{-1}(t)}^{Def} \lambda\left(\mathbf{w}^{(t)}, m^\star\right), \quad m^\star < m \\
&(\text{Euclidean projection}) && \mathbf{w}^{(t)} = \boldsymbol{\theta}^{(t-1)} + \mathbf{\Phi}^T\left(\mathbf{\Phi}\mathbf{\Phi}^T\right)^{-1}\left(\mathbf{y} - \mathbf{\Phi}\boldsymbol{\theta}^{(t-1)}\right) \\
&(\text{groupwise shrinkage}) && \boldsymbol{\theta}_{\mathcal{G}_k}^{(t)} = \mathbf{w}_{\mathcal{G}_k}^{(t)} \max\left\{1 - \lambda(\mathbf{w}^{(t)}, m^\star)\left\|\mathbf{w}_{\mathcal{G}_k}^{(t)}\right\|_2^{-1}\beta_k, 0\right\} \quad \forall k \in \mathbb{N}_m
\end{aligned}$$

其中关于 λ的更新较为复杂，详细推导见原论文。

交替方向乘子法 (Alternating direction method of multipilers, ADMM)

考虑优化问题:

$$\begin{aligned}
\min_{x,z} \quad & f(\mathbf{x}) + g(\mathbf{z}) \\
\text{s.t.} \quad & \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{z} - \mathbf{c} = 0
\end{aligned}$$

ADMM的算法流程为:

$$\begin{aligned}
\mathbf{x}^{k+1} &= \arg\min_{\mathbf{x}} f(\mathbf{x}) + \frac{\rho}{2}\left\|A\mathbf{x} + B\mathbf{z}^k - \mathbf{c} + \boldsymbol{\mu}^k\right\|_2^2 \\
\mathbf{z}^{k+1} &= \arg\min_{\mathbf{z}} g(\mathbf{z}) + \frac{\rho}{2}\left\|A\mathbf{x}^{k+1} + B\mathbf{z} - \mathbf{c} + \boldsymbol{\mu}^k\right\|_2^2, \quad \left(\boldsymbol{\mu}^k := \frac{\boldsymbol{\lambda}^k}{\rho}\right) \\
\boldsymbol{\mu}^{k+1} &= \boldsymbol{\mu}^k + \left(A\mathbf{x}^{k+1} + B\mathbf{z}^{k+1} - \mathbf{c}\right)
\end{aligned}$$

关于ADMM的推导过程建议参考 ADMM算法的详细推导过程是什么？ - 覃含章的回答

关于在实际场景中如果将问题转换为ADMM形式来求解，推荐参考 交替方向乘子法（ADMM）算法的流程和原理是怎样的？ - 门泊东吴的回答，下面摘录其中一例，求解含有 l_1范数的优化问题:

$$\min_{x,z} \quad l(\mathbf{x}) + \lambda\|\mathbf{x}\|_1$$

s首先引入辅助变量 $\mathbf{z}$ 将上述问题转换为 ADMM 可以求解的形式:

$$\begin{aligned}
\min_{x,z} \quad & l(\mathbf{x}) + \lambda\|\mathbf{z}\|_1 \\
\text{s.t.} \quad & \mathbf{x} - \mathbf{z} = \mathbf{0}
\end{aligned}$$

使用ADMM的求解流程为:

$$\begin{aligned}
&\mathbf{x}^{k+1} = \arg\min_{\mathbf{x}} l(\mathbf{x}) + \tfrac{\rho}{2}\left\|\mathbf{x} - \mathbf{z}^k + \boldsymbol{\mu}^k\right\|_2^2 \\
&\mathbf{z}^{k+1} = S_{\lambda/\rho}\left(\mathbf{x}^{k+1} + \boldsymbol{\mu}^k\right) \\
&\boldsymbol{\mu}^{k+1} = \boldsymbol{\mu}^k + \left(\mathbf{x}^{k+1} - \mathbf{z}^{k\perp 1}\right) \\
&\mathrm{S}_\kappa(a) := \begin{cases} a - \kappa, & a > \kappa \\ 0, & |a| \leq \kappa \\ a + \kappa, & a < -\kappa \end{cases}
\end{aligned}$$

其中 $\mathrm{S}_\kappa(a)$ 是软阈值函数。对于LASSO问题，上述 $l(x) = \frac{1}{2}\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2$，则 $\mathbf{x}^{k+1}$的解析解可以写成:
$\mathbf{x}^{k+1} = (\mathbf{A}^\top\mathbf{A} + \rho I)^{-1}(\mathbf{A}^\top\mathbf{b} + \rho(\mathbf{z}^k - \mu^k))$

算子分裂二次规划求解算法 (Operator splitting solver for quadratic programs, OSQP)

考虑一般的（凸）二次规划（quadratic program，QP）问题:

$$\begin{aligned}
\min_{x} \quad & \frac{1}{2}\mathbf{x}^\top\mathbf{P}\mathbf{x} + \mathbf{q}^\top\mathbf{x} \\
\text{s.t.} \quad & \mathbf{A}\mathbf{x} \in \mathcal{C}
\end{aligned}$$

其中$\mathbf{x} \in \mathbf{R}^n, \mathbf{P} \in \mathbf{S}_+^n, \mathbf{q} \in \mathbf{R}^n; \mathbf{A} \in \mathbf{R}^{m\times n}, \mathcal{C} \subseteq \mathbf{R}^m$

如果集合 $\mathcal{C}=[l,u]=\{z\in\mathbf{R}^m|l_i\le z_i\le u_i,i=1,\cdots,m\}$，其中 $l_i\in\{-\infty\}\cup\mathbf{R},u_i\in\mathbf{R}\cup\{+\infty\}$，则上述问题转化为：

$$\begin{aligned}\min_x \quad & \frac{1}{2}\mathbf{x}^\top\mathbf{P}\mathbf{x}+\mathbf{q}^\top\mathbf{x}\\ \text{s.t.} \quad & l\le\mathbf{A}\mathbf{x}\le u\end{aligned}$$

当 $l_i=u_i$对所有元素均成立，或者对部分元素成立时，相应约束即变为等式约束。

将上述问题转换为可以利用ADMM算法求解的形式：

$$\begin{array}{ll}\text{minimize} & (1/2)\tilde{x}^TP\tilde{x}+q^T\tilde{x}+\mathcal{I}_{Ax=z}(\tilde{x},\tilde{z})+\mathcal{I}_{\mathcal{C}}(z)\\ \text{subject to} & (\tilde{x},\tilde{z})=(x,z)\end{array}$$

其中：

$$\mathcal{I}_{Ax=z}(x,z)=\begin{cases}0 & Ax=z\\ +\infty & \text{otherwise}\end{cases},\quad \mathcal{I}_{\mathcal{C}}(z)=\begin{cases}0 & z\in\mathcal{C}\\ +\infty & \text{otherwise}\end{cases}$$

利用ADMM算法可以将上述优化问题分为三步迭代求解：

$$\begin{aligned}(\tilde{x}^{k+1},\tilde{z}^{k+1})\leftarrow & \underset{(\tilde{x},\tilde{z}):A\tilde{x}=\tilde{z}}{\operatorname{argmin}}\ (1/2)\tilde{x}^TP\tilde{x}+q^T\tilde{x}+(\sigma/2)\|\tilde{x}-x^k+\sigma^{-1}w^k\|_2^2 \qquad (15)\\ & \qquad\qquad +(\rho/2)\|\tilde{z}-z^k+\rho^{-1}y^k\|_2^2\\ x^{k+1}\leftarrow & \alpha\tilde{x}^{k+1}+(1-\alpha)x^k+\sigma^{-1}w^k \qquad (16)\\ z^{k+1}\leftarrow & \Pi\left(\alpha\tilde{z}^{k+1}+(1-\alpha)z^k+\rho^{-1}y^k\right) \qquad (17)\\ w^{k+1}\leftarrow & w^k+\sigma\left(\alpha\tilde{x}^{k+1}+(1-\alpha)x^k-x^{k+1}\right) \qquad (18)\\ y^{k+1}\leftarrow & y^k+\rho\left(\alpha\tilde{z}^{k+1}+(1-\alpha)z^k-z^{k+1}\right) \qquad (19)\end{aligned}$$

其中 $\sigma>0,\rho>0$为步长参数，$\alpha\in(0,2)$是松弛参数，Π表示向$\mathcal{C}$集合上的欧几里得投影（Euclidean projection）。根据 (16) 和 (18) 可以推出来$w^{k+1}=0,\forall k>0$，所以表示 w的迭代更新的 (18) 式可以去掉。上述迭代的关键步骤在于 (15) 式，通过KKT条件可以推出其解析解为：

$$\begin{bmatrix}P+\sigma I & A^T\\ A & -\rho^{-1}I\end{bmatrix}\begin{bmatrix}\tilde{x}^{k+1}\\ \nu^{k+1}\end{bmatrix}=\begin{bmatrix}\sigma x^k-q\\ z^k-\rho^{-1}y^k\end{bmatrix}$$

$$\tilde{z}^{k+1}=z^k+\rho^{-1}(v^{k+1}-y^k)$$

具体推导见原论文。OSQP算法的迭代步骤具体如下：

Algorithm 1

1: **given** initial values x^0, z^0, y^0 and parameters $\rho>0$, $\sigma>0$, $\alpha\in(0,2)$
2: **repeat**
3: $\quad(\tilde{x}^{k+1},\nu^{k+1})\leftarrow$ solve linear system $\begin{bmatrix}P+\sigma I & A^T\\ A & -\rho^{-1}I\end{bmatrix}\begin{bmatrix}\tilde{x}^{k+1}\\ \nu^{k+1}\end{bmatrix}=\begin{bmatrix}\sigma x^k-q\\ z^k-\rho^{-1}y^k\end{bmatrix}$
4: $\quad\tilde{z}^{k+1}\leftarrow z^k+\rho^{-1}(\nu^{k+1}-y^k)$
5: $\quad x^{k+1}\leftarrow\alpha\tilde{x}^{k+1}+(1-\alpha)x^k$
6: $\quad z^{k+1}\leftarrow\Pi\left(\alpha\tilde{z}^{k+1}+(1-\alpha)z^k+\rho^{-1}y^k\right)$
7: $\quad y^{k+1}\leftarrow y^k+\rho\left(\alpha\tilde{z}^{k+1}+(1-\alpha)z^k-z^{k+1}\right)$
8: **until** termination criterion is satisfied

上述迭代步骤中，除了步骤3之外都很简单，因为它们仅包含向量加减，数乘以及投影操作，而且都是 component-wise separable 的操作，能够很容易的进行并行。下面以LASSO问题为例，将其转换为可以用OSQP求解的二次规划问题形式：

LASSO问题的一般形式为：

$$\min_x \quad \|\mathbf{A}\mathbf{x}-\mathbf{b}\|^2+\lambda\|\mathbf{x}\|_1$$

其中,$\mathbf{x}\in\mathbf{R}^n$是参数向量，$\mathbf{A}\in\mathbf{R}^{m\times n}$ 是数据矩阵，λ是权重参数。将该问题转换为二次规划问题：

$$
\begin{array}{ll}
\min & \mathbf{y}^{\top}\mathbf{y}+\lambda\mathbf{1}^{\top}t \\
\text{s.t.} & \mathbf{y}=\mathbf{A}\mathbf{x}-\mathbf{b} \\
 & -\mathbf{t}\leq\mathbf{x}\leq\mathbf{t}
\end{array}
$$

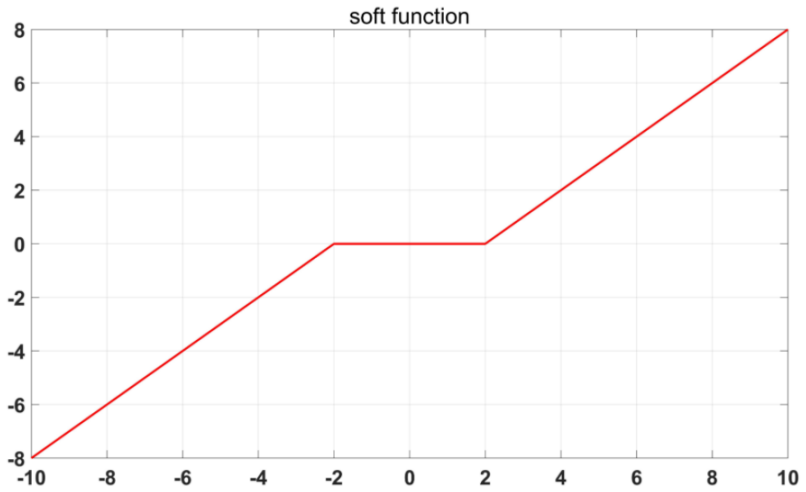
其中 $\mathbf{y}\in\mathbf{R}^m,\mathbf{t}\in\mathbf{R}^n$ 是两个新引入的变量。转化为二次规划形式后的LASSO问题即可以用上述OSQP算法求解。

附录 Appendix

软阈值算子（soft-thresholding operator）

表达式 $\mathrm{soft}_T(x)=sign(x)(|x|-T)_+=sign(x)\max(|x|-T,0)=$,

图像如下：



Lipschitz常数和Lipschitz连续

利普希茨连续的定义是：如果函数f在区间$\mathcal{Q}$上以常数L利普希茨连续，那么对于$x,y\in\mathcal{Q}$，有：

$$\|f(x)-f(y)\|\leq L\|x-y\|$$

其中常数L称为f在区间$\mathcal{Q}$上的Lipschitz常数。除了Lipschitz continuous之外，Lipschitz continuous gradient 和 Lipschitz continuous Hessian也是常用到的概念，它们都是由Lipschitz continuous概念延伸出来的。值得一提的是，很多论文中，尤其是关于凸优化的问题，**Lipschitz continuous gradient**的应用更为常见。

其中，如果函数f满足Lipschitz continuous gradient，就意味着它的导数f'满足Lipschitz continuous，即如果函数f满足Lipschitz continuous gradient，则：

$$\|f'(x)-f'(y)\|\leq L\|x-y\|$$

其中，如果函数f满足Lipschitz continuous Hessian，就意味着它的二阶导数f''满足Lipschitz continuous，即如果函数f满足Lipschitz continuous Hessian，则：

$$\|f''(x)-f'(y)\|\leq L\|x-y\|$$

从上面的定义中可以看出，利普希茨连续限制了函数f的局部变动幅度不能超过某常量。同样的道理，Lipschitz continuous gradient和Lipschitz continuous Hessian则限制了函数的导函数和二阶导函数的局部变化幅度。

根据上述第二种Lipschitz continuous gradient条件可以有以下定理：

如果函数f在$\mathbb{R}^n$上满足Lipschitz continuous gradient，则对于任意$x,y\in\mathbb{R}^n$ 有：

$$|f(y)-f(x)-<f'(x),y-x>|\leq\frac{L}{2}\|y-x\|^2$$

根据上面的公式，去掉绝对值可以得到等价表达：

$$
\begin{cases}
f(y)\leq f(x)+<f'(x),y-x>+\frac{L}{2}\|y-x\|^2 \\
f(y)\geq f(x)+<f'(x),y-x>-\frac{L}{2}\|y-x\|^2
\end{cases}
$$

其余两种Lipschitz continuous条件也有相应的定理，具体定理与证明见论文。

Nesterov加速技术

Nesterov加速技术由大神Yurii Nesterov (内斯特罗夫) 于1983年提出来的，它与目前深度学习中用到的经典的动量方法（Momentum method）很相似，和动量方法的区别在于二者用到了不同点的梯度，动量方法采用的是上一步迭代点的梯度，而Nesterov方法则采用从上一步迭代点处朝前走一步处的梯度。在几乎所有与梯度有关的算法中，Nesterov加速技术都可以使用。具体对比如下。

动量方法：

$$
\begin{aligned}
v_{t+1} &= u_t v_t - \alpha_t \nabla g(\theta_t) \\
\theta_{t+1} &= \theta_t + v_{t+1}
\end{aligned}
$$

Nesterov 方法：

$$
\begin{aligned}
v_{t+1} &= u_{t+1} v_t - \alpha_t \nabla g(\theta_t + u_t v_t) \\
\theta_{t+1} &= \theta_t + v_{t+1}
\end{aligned}
$$

详见Nesterov加速和Momentum动量方法

Reference

- HQS——Half Quadratic Splitting半二次方分裂
- 近端梯度下降与软阈值迭代：PGD and ISTA
- 软阈值迭代算法（ISTA）和快速软阈值迭代算法（FISTA）
- LASSO回归与L1正则化 西瓜书
- 【优化】利普希茨连续（Lipschitz continuous）及其应用
- FISTA的由来：从梯度下降法到ISTA & FISTA
- Nesterov加速和Momentum动量方法
- IST改进算法之Two-Step Iterative Shrinkage/Thresholding(TwIST)
- 交替方向乘子法（ADMM）算法的流程和原理是怎样的？
- ADMM算法的详细推导过程是什么？ - 覃含章的回答
- 交替方向乘子法（ADMM）算法的流程和原理是怎样的？ - 门泊东吴的回答
- Beck, Amir, and Marc Teboulle. “A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems.” *SIAM Journal on Imaging Sciences* 2, no. 1 (January 2009): 183–202. https://doi.org/10.1137/080716542.
- Bioucas-Dias, J.M., and M.A.T. Figueiredo. “A New TwIST: Two-Step Iterative Shrinkage/Thresholding Algorithms for Image Restoration.” *IEEE Transactions on Image Processing* 16, no. 12 (December 2007): 2992–3004. https://doi.org/10.1109/TIP.2007.909319.
- Zhou, Xingyu. “On the Fenchel Duality between Strong Convexity and Lipschitz Continuous Gradient.” *ArXiv:1803.06573 [Math]*, March 17, 2018. http://arxiv.org/abs/1803.06573.
- Liao, Xuejun, Hui Li, and Lawrence Carin. “Generalized Alternating Projection for Weighted-$\ell_{2,1}$ Minimization with Applications to Model-Based Compressive Sensing.” *SIAM Journal on Imaging Sciences* 7, no. 2 (January 2014): 797–823. https://doi.org/10.1137/130936658.
- Stellato, Bartolomeo, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. “OSQP: An Operator Splitting Solver for Quadratic Programs.” *Mathematical Programming Computation* 12, no. 4 (December 2020): 637–72. https://doi.org/10.1007/s12532-020-00179-2.